

《高级程序设计》全真模拟卷 — 参考答案与解析

所有程序输出均经 g++ 13 实际编译运行验证（笔试惯例：-std=c++17 -fno-elide-constructors）。红框为你分析报告中的高频失分点。

一、简答题

1. 函数重写 (override) 与函数重载 (overload) 的区别

- **重载 overload**：同一作用域内，函数名相同、参数列表不同（个数 / 类型 / 顺序）。编译期按实参静态选择；与返回值无关；不需要继承。
- **重写 override**：派生类重新定义基类的虚函数，函数签名相同（virtual+ 相同名与参数）。运行期通过基类指针 / 引用动态绑定选择；必须有继承 + virtual。

一句话：重载 = 同名不同参 · 编译期 · 同一类；重写 = 同签名 · 运行期 · 父子类 + virtual。

例：重载 `int f(int); double f(double);`；重写 基类 `virtual void area();` → 派生类 `void area()` override;

你的弱点①：多道题把 override 说成"重载"。本卷 Q2、Q3 凡涉及虚函数务必写"**重写 / override / 动态绑定**"。

2. 纯虚函数与抽象类

- **纯虚函数**：只声明不实现的虚函数，语法 `virtual 返回类型 函数名(参数) = 0;`
- **抽象类**：含至少一个纯虚函数的类。不能实例化（不能创建对象），只能作基类被继承。
- **作用**：为派生类提供统一的对外接口 / 框架，强制派生类实现这些接口，从而支持多态。
- **接口类**：所有成员函数都是纯虚函数（且带虚析构函数）的抽象类。

你的弱点②：真题 Q4 接口类几乎空白。记住三要素：**全是纯虚函数 + 虚析构 + 不可实例化**。

3. 深拷贝 vs 浅拷贝；何时必须自定义拷贝构造

- **浅拷贝**：逐成员复制；若成员是指针，只复制指针值 → 两对象指向同一块内存。
- **深拷贝**：为指针成员另行 new 内存并复制内容 → 两对象各自独立。
- **必须自定义（深拷贝）的情形**：类持有裸指针 / 动态分配的资源（new 的内存、文件句柄等）。否则默认浅拷贝会让两对象共享资源，析构时 `double free` / 悬空指针。
- **不需要**：成员只有 `string` / `vector` / `int` 等自带深拷贝语义的类型时，编译器默认拷贝构造就够了。

配套"三法则"：需要自定义 析构 / 拷贝构造 / 拷贝赋值 之一时，通常三者都要写。

你的弱点③：真题里你误以为含 `string` 的 `Student` 也会 `double free`。**无裸指针就不会**——见本卷 Q1 第③问。

4. 友元 (friend) 函数

- **定义**：用 `friend` 声明、可访问该类 `private` / `protected` 成员的非成员函数（或别的类 / 其成员函数）。
- **与成员函数区别**：友元不是成员，没有 this 指针，调用形如 `f(obj)` 而非 `obj.f()`；成员函数有 `this`、通过对象调用。
- **为何引入**：提高对私有数据的访问效率与灵活性。典型场景：重载 `<<` / `>>`（左操作数是 `ostream` 而非本类对象，无法写成成员函数）。

二、程序分析题

Q1 (构造 / 拷贝 / 析构追踪)

① 完整输出:

```
===1===
Vehicle(BMW) ctor
Engine(L4) ctor
Car(BMW) ctor
===2===
Vehicle(Tesla) ctor
Engine(V8) ctor
Car(Tesla) ctor
Vehicle(Tesla) copy
Engine(V8) copy
Car copy
Car dtor
Engine(V8) dtor
Vehicle(Tesla) dtor
===3===
Vehicle(BMW) copy
Engine(L4) copy
Car copy
-- in test --
Car dtor
Engine(L4) dtor
Vehicle(BMW) dtor
===4===
Car dtor
Engine(V8) dtor
Vehicle(Tesla) dtor
Car dtor
Engine(L4) dtor
Vehicle(BMW) dtor
```

② 拷贝构造的位置:

- `makeCar()` 中 `return c`; —— 返回时把局部 `c` 拷贝到返回结果 (1 次)。
- `test(c1)` —— 值传参, 把 `c1` 拷贝给形参 (1 次)。
- `Car c2 = makeCar();` 这条语句本身在 C++17 下 不再额外拷贝 (返回的临时量直接初始化 `c2`)。

共 2 次 `Car` 拷贝, 每次都连带触发 `Vehicle`、`Engine` 的拷贝构造。

③ **删除自定义拷贝构造后: 程序仍能正常运行。** `Car` 没有裸指针成员 (只有基类 `Vehicle` 和对象成员 `Engine` / `string`), 编译器默认拷贝构造做逐基类 / 逐成员拷贝, 不会 `double free`。变化: 原来打印 "`Car copy`" 的地方不再打印 (默认拷贝构造没有打印语句), 但 `Vehicle(...)` `copy`、`Engine(...)` `copy` 仍会打印——因为默认拷贝构造会调用基类和成员的拷贝构造。

Q2 (虚函数与多态)

① 输出:

```
I say: Dog: Woof
I say: Cat: Meow
Dog: Woof
```

② `intro()` 虽非虚函数, 但其内部调用的 `sound()` 是 虚函数; 通过隐含的 `this` 指针调用 `sound()` 时发生 **动态绑定 (动态分派)**, 运行期根据 `this` 实际指向的对象 (`Dog` / `Cat`) 选择对应的 `sound()`。这是 "**虚函数重写 (override) + 动态绑定**", 不是重载。

③ 去掉 `virtual` 后变为 **静态绑定**, 编译期按指针的静态类型 `Animal*` 决定调用 `Animal::sound()`。输出变为:

```
I say: Animal: ...
I say: Animal: ...
Animal: ...
```

Q3 (运算符重载)

① 输出:

```
c = (4, 6)
a == b ? false
a = (2, 3), d = (2, 3)
a + b == c ? false
```

(最后一行: ++a 后 a=(2,3), a+b=(5,7)≠c=(4,6), 故 false。)

② operator<< 返回 ostream&, 使多个 << 能链式串联 (左结合, 前一个 << 的结果作下一个的左操作数)。若返回类型改成 void: cout << "c = " 得到 void, 再 << c 无法匹配 → **编译错误**, 链式输出失效。

你的弱点④: 流运算符习惯性漏写 return os; ——这正是本题第②问的考点。

③ 支持 a * 2.0 的重载:

```
// 成员函数版 (支持 a * 2.0)
Vector2D operator*(double k) const {
    return Vector2D(x * k, y * k);
}
// 若还要支持 2.0 * a, 再加一个非成员/友元函数:
friend Vector2D operator*(double k, const Vector2D& v) {
    return Vector2D(v.x * k, v.y * k);
}
```

Q4 (异常处理)

① 输出 (无 throw;):

```
middle caught: divide by zero
middle: done
main: ok
Final stage = 7
```

stage = 4; **不会执行**: inner(0) 抛异常后, 控制权直接跳到 middle 的 catch, try 块内 inner(x) 之后的语句 (stage=4, "middle: ok") 被跳过。但 catch 结束后**没有重抛**, 程序**继续执行** try/catch 之后的语句, 所以 "middle: done"、"main: ok" 都会打印, **stage 最终 = 7**。

你的弱点⑤: 真题里你漏掉了 catch 后继续执行的那行 ("middle: done" 类)。记牢: **catch 完且不重抛 → 接着往下跑**。

② 在 catch 末尾加 throw; 后:

```
middle caught: divide by zero
main caught: divide by zero
Final stage = 8
```

机制改变: throw; 重新抛出当前异常 → middle 中 catch 之后的代码 (stage=6, "middle: done") 被跳过, 异常继续向上**栈展开**传播到 main 的 catch 捕获, 故打印 "main caught: ...", "main: ok" (stage=7) 被跳过, **stage 最终 = 8**。

三、编程题

编程 1: Complex 复数类

参考实现 (类外定义):

```

Complex::Complex(double r, double i) : re(r), im(i) {}

Complex Complex::operator+(const Complex& rhs) const {
    return Complex(re + rhs.re, im + rhs.im);
}
Complex Complex::operator-(const Complex& rhs) const {
    return Complex(re - rhs.re, im - rhs.im);
}
bool Complex::operator==(const Complex& rhs) const {
    return re == rhs.re && im == rhs.im;
}
ostream& operator<<(ostream& os, const Complex& c) {
    os << c.re;
    if (c.im >= 0) os << "+" << c.im << "i";
    else          os << "-" << -c.im << "i";
    return os;
    // * 必须 return os, 否则无法链式输出
}

```

运行输出 (已验证):

```

3+4i
1-2i
0+0i
5+0i
4+2i
2+6i
true
false
-1+2i

```

易错点: ① operator<< 必须 return os;; ② operator+/- 返回类型是 Complex (不是 double); ③ 类外定义构造函数时初始化列表写 Complex::Complex(double r,double i):re(r),im(i){}, 默认参数只写在声明处。

编程 2: RefPtr 智能指针 + DataPool 链表容器

参考实现 (仅 TODO 部分):

```

// 1.1 拷贝构造: 浅拷贝 + 计数加一
RefPtr(const RefPtr& other) : ptr(other.ptr), cnt(other.cnt) {
    if (cnt) cnt->n++;
    // 引用参数用 . 不用 ->
}

// 1.2 析构: 只调一次 release(), 不要再手动 delete (否则 double free)
~RefPtr() { release(); }

// 1.3 赋值运算符 (四步缺一不可)
RefPtr& operator=(const RefPtr& other) {
    if (this != &other) {
        release();
        ptr = other.ptr;
        cnt = other.cnt;
        if (cnt) cnt->n++;
    }
    return *this;
}

// 1.4 解引用
T& operator*() const {
    assert(ptr != nullptr);
    return *ptr;
}

// 2.1 登记新对象 (头插法)
void add(T* raw) {

```

```

    RefPtr<T> ref(raw);
    PoolNode<T>* node = new PoolNode<T>(ref);
    node->next = head;
    head = node;
    count++;
}

// 2.2 map_to: 遍历链表, 逐个映射
template<typename U, typename Func>
vector<U> map_to(Func f) const {
    vector<U> result;
    PoolNode<T>* cur = head;
    while (cur) {
        if (cur->data.valid())
            result.push_back(f(*(cur->data)));
        cur = cur->next;
    }
    return result;
}

// 2.3 select: 遍历链表, 满足 pred 则收集副本
vector<T> select(function<bool(const T&)> pred) const {
    vector<T> result;
    PoolNode<T>* cur = head;
    while (cur) {
        if (cur->data.valid() && pred(*(cur->data)))
            result.push_back(*(cur->data));
        cur = cur->next;
    }
    return result;
}

```

运行输出 (已验证):

```

Item X born
count = 2
count = 3
access: X
Item X gone
[freed]
-----
Item A born
Item B born
Item C born
[ {C:5} {B:8} {A:3} ]
map: 50 80 30
Item C gone
select: {C:5} {B:8}
Item C gone
Item B gone
Item C gone
[freed]
Item B gone
[freed]
Item A gone
[freed]

```

输出要点: count=2 → count=3; 离开作用域时引用计数归 0 打印 [freed]; show() 因头插而逆序输出 [{C:5} {B:8} {A:3}]; map: 50 80 30; select: {C:5} {B:8}。注意 select 返回的是对象副本, 所以会额外触发 Item 的拷贝与析构 (多出几行 Item ... gone)。

你的弱点⑥⑦: ① 析构只调一次 release(), **不要再手动 delete**; ② operator= 四步: 自赋值检测 → 释放旧 → 接管并 ++ → return *this; ③ 引用参数用 . 不用 ->; ④ 链表遍历牢记 while(cur){ ... cur=cur->next; }。

今晚冲刺清单

1. **术语**：虚函数场景一律写"重写 / override / 动态绑定 / 静态绑定"，杜绝"重载"。
2. **接口类**：纯虚函数 =0 语法 + 虚析构 + 不可实例化，能默写一段最小接口类代码。
3. **operator=** 四步、**operator<<** 的 return os、**三法则**。
4. **构造 / 析构顺序**：基类 → 成员对象 → 派生类；析构相反。return 局部对象 = 1 次拷贝。
5. **异常**：catch 不重抛会继续往下执行；throw；重抛会跳过后续并栈展开。
6. **链表三件套**：头插、while 遍历、map/filter。